

南開大學
Nankai University

南　开　大　学

计 算 机 学 院

编译器课程预备工作实验报告

编译流程与多层中间表示

SysY、LLVM IR、RISC-V 与 MLIR

柯云超　2413575

郑奕杰　2412677

代码仓库：https://github.com/kyc001/compiler

小组分工安排：

- **柯云超：**负责实验总体方案与技术路线设计，主导 SysY、LLVM IR 和 RISC-V 汇编的核心实现与语义等价验证；完成 SSA 与优化效果分析、MLIR 逐层降低实验及 AscendNPU IR 调研，构建自动化验证流程，统筹报告的技术分析与结论论证。
- **郑奕杰：**负责阶乘样例的预处理、编译、汇编与链接基础实验，整理各阶段命令及输出；执行基础与边界用例，核对预期结果，协助整理实验环境说明、结果图表和参考资料。
- 共同工作：交叉检查实验代码、核对结果与引用，复核并提交同一份报告。

2026 年 10 月 10 日

摘要

本实验以阶乘程序研究预处理、编译、汇编和链接的完整过程，观察宏替换、前端诊断、SSA 提升、机器指令、符号及重定位，并比较优化等级、调试选项与静态链接的影响。独立编写三组 SysY、LLVM IR 与 RISC-V64 等价程序，覆盖短路求值、作用域、数组、递归、整数除余及浮点运算与转换；34 个输入在五条执行路径上完成 170 次比对，全部通过，阶乘的五种构建结果另通过 85 次检查。进阶实验实测通用 MLIR 向量加的逐层降低，以三组输入／输出布局验证 12 个元素，并结合官方 VecAdd 分析 AscendNPU IR 的方言职责与数据依赖。结果表明，低层表示必须显式维护控制流、数据布局与调用约定，文件体积不能直接代表性能。Ascend 专用部分采用文档调研方式，未执行 BishengIR/CANN 或 NPU 程序。

关键词：编译流程；SysY；LLVM IR；RISC-V；MLIR

目录

一、 引言 **1**

二、 实验环境与设计 **1**

（一） 工具链和运行环境 1

（二） 实验对象与验证口径 1

三、 完整编译流程实验 **2**

（一） 程序与阶段输出 2

（二） 预处理、前端分析与语义错误 3

（三） LLVM IR 与 SSA 提升 3

（四） 优化等级与调试选项的对照 4

（五） 汇编、符号与链接 4

四、 SysY、LLVM IR 与 RISC-V 等价编程 **5**

（一） 样例设计与语言特性 5

（二） 控制流、短路与作用域 5

（三） 整数语义与 RISC-V 调用约定 6

（四） 数组地址与浮点转换 6

（五） 负数运算、递归与数组写入的补充验证 7

（六） 链接路径及结果 7

五、 MLIR 与 AscendNPU IR 探索 **8**

（一） 通用 VecAdd 的逐层降低 8

（二） 官方 Ascend VecAdd 与方言分工 9

（三） 从官方示例推导存储与同步约束 9

（四） 比较与探索边界 10

六、 结论 **11**

附录 A 复现与结果文件 **11**

参考文献 **12**

一、 引言

编译器将具有类型、作用域和结构化控制流的源程序转换为机器可执行代码。仅观察最终输出，难以区分前端检查、中端优化、后端生成以及汇编链接各自承担的工作。本实验围绕课程预备要求 [1]，以一个阶乘程序研究完整语言处理过程，再以三组 SysY 程序验证手写 LLVM IR 和 RISC-V 汇编的行为，最后探索 MLIR 多级表示。

实验按“源程序或编译选项发生变化，阶段输出如何变化”的方法组织。基础概念结合实际输出说明，重点分析短路副作用、SSA、数组地址计算、浮点转换及多级降低。报告中的数据来自同一批自动化实验；保存的结果、命令和关键输出位于 `lab0/report/evidence/`，可通过 `make prep` 复现。测试验证给定输入上的一致性，不构成任意程序上的等价性证明。

二、 实验环境与设计

(一) 工具链和运行环境

实验在 Ubuntu 22.04 的 WSL 环境完成，宿主架构为 x86_64。表1列出实际使用的工具。RISC-V 程序以 QEMU 用户模式执行，使用 RV64GC 指令集和 LP64D ABI。

表 1: 实际使用的主要工具

工具或配置	版本或参数
GCC	Ubuntu GCC 12.3.0
Clang / LLVM / MLIR	15.0.7
RISC-V GCC	SiFive GCC-Metal 10.2.0
QEMU 用户模式	6.2.0
RISC-V 链接	`-static -mcmodel=medany`
地址与重定位选项	`-Wl,--no-relax,-Ttext=0x90000000`

运行库来自课程提供的 `lab0/runtime/lib.tar.gz`[3, 4]。宿主程序链接其中的 `libsysy_x86.a`。预编译 RISC-V 库的 ISA 属性与本机 GCC 10 链接器不兼容，因此按讲义允许的方法，从未经修改的 `sylib.c` 重建兼容库。课程头文件定义了计时全局变量，C 参考程序采用 `-fcommon` 编译；标准错误上的运行库计时信息单独记录，功能输出按标准输出比较。

(二) 实验对象与验证口径

阶乘程序用于编译流程实验，合法输入为 $0 \le n \le 12$；$12! = 479001600$ 仍在 32 位有符号整数范围内，$13!$ 越界，因此先检查范围。等价编程使用 `features`、`arrays_float` 和 `numeric_edges`，分别测试控制流、二维浮点数组及数值边界。通过强制包含课程头文件补充运行库声明，建立 C 编译器可处理的参考路径；它不依赖尚未实现完整功能的课程编译器。SysY 浮点字面量为 float，C 中无后缀字面量为 double；本实验选取二进制可精确表示的常量及有界输入，各浮点中间结果也可精确表示，使这一差异不影响所测结果，不能由此断言二者对任意源码兼容。

预期值保存在独立用例文件中。每次执行同时检查退出码和标准输出，保留错误诊断；编译及运行失败会中止脚本。IR 先经过 LLVM verifier，随后才进入目标生成和执行。文件大小、函

数符号大小与测试次数采用不同口径，避免将调试信息体积误当作指令数量。

三、 完整编译流程实验

(一) 程序与阶段输出

主程序用 scanf 读入并检查整数，调用阶乘函数后用 printf 输出。其核心代码如下：

```c
#ifndef FIRST_FACTOR
#define FIRST_FACTOR 2
#endif
int factorial(int n) {
    int result = 1;
    int i = FIRST_FACTOR;
    while (i <= n) {
        result = result * i;
        i = i + 1;
    }
    return result;
}
```

图1说明处理过程，表2对应可检查的产物。图中编译器内部各步骤是功能划分，具体工具可能将多个步骤融合实现。

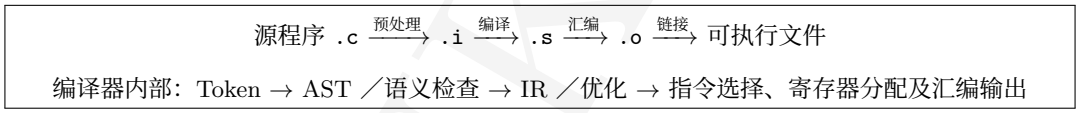


图 1: 语言处理系统及编译器内部的功能分工

表 2: 阶段、观察命令及对应信息

阶段	主要选项	观察内容
预处理	`gcc -E`	宏与头文件展开、来源行标记
词法	`clang-15 -Xclang -dump-tokens`	Token 类型、文本及位置
语法／语义	`-Xclang -ast-dump`、`-fsyntax-only`	AST、类型与未声明标识符诊断
中间代码	`-fdump-tree-gimple`、`-emit-llvm`	显式分支、基本块与数据操作
优化／后端	`opt-15 -passes=mem2reg`、`gcc -S`	SSA、指令和寄存器使用
汇编	`gcc -c`、`objdump -dr`	机器码与重定位位置
链接	`gcc factorial.o -o factorial`	符号解析、节布局与库依赖

以下命令概括基本处理链；自动化脚本使用完整路径，并保留其参数 [5]。

```
gcc -E factorial.c -o factorial.i
gcc -S -O0 -g factorial.i -o factorial_O0.s
gcc -c factorial_O0.s -o factorial.o
gcc factorial.o -o factorial
```

(二) 预处理、前端分析与语义错误

预处理器展开 `#include` 和宏、处理条件编译，消除注释并输出来源行标记。本实验中 `FIRST_FACTOR` 在 `.i` 中被替换为 2；添加 `-DFIRST_FACTOR=3` 后变为 3。输入 5 时，默认程序计算 $2 \times 3 \times 4 \times 5 = 120$，修改后的程序输出 $3 \times 4 \times 5 = 60$。宏改变了后续编译器看到的程序，预处理器本身不负责类型检查或生成机器码。

词法分析把 `while (i <= n)` 分解为关键字、括号、标识符和关系运算符等 Token，语法分析再建立 `WhileStmt`、条件表达式和循环体等 AST 节点。语义分析进一步完成名称绑定、类型检查和隐式转换。单独编译“返回未声明的 `missing_name`”的函数时，Clang 报告 `use of undeclared identifier 'missing_name'`，退出码为 1。程序的括号和语句结构合法仍不代表可以进入代码生成。

GCC 的 GIMPLE 输出把 `while` 改写为标签和跳转。实际转储中的核心语句为:

```
result = 1; i = 2;
goto <D.2386>;
<D.2387>:
result = result * i;
i = i + 1;
<D.2386>:
if (i <= n) goto <D.2387>; else goto <D.2385>;
```

源语言的循环嵌套在此变为控制流关系，后续分析便可围绕基本块和数据依赖进行。

(三) LLVM IR 与 SSA 提升

使用 `-O0 -Xclang -disable-O0-optnone` 生成 LLVM IR，再单独执行 `mem2reg`，以隔离这一变换。禁用 `optnone` 是为了允许优化 pass 处理 O0 函数 [6]。

```
clang-15 -S -emit-llvm -O0 -Xclang -disable-O0-optnone \
  factorial.c -o factorial.raw.ll
opt-15 -S -passes=mem2reg factorial.raw.ll \
  -o factorial.ssa.ll
```

整个模块的 `alloca` 从 5 条降为 1 条，出现 3 条 `phi`。其中两条处理阶乘循环的累乘值和循环变量，另一条合并 main 的返回值。实际优化结果中的循环头如下，数值标签和变量名与转储一致:

```
2:  ; predecessors: %4, %1
  %.01 = phi i32 [ 1, %1 ], [ %5, %4 ]
  %.0 = phi i32 [ 2, %1 ], [ %6, %4 ]
  %3 = icmp sle i32 %.0, %0
  br i1 %3, label %4, label %7
```

phi 根据控制流前驱选择初始值或上轮结果，使每个 SSA 名只定义一次。main 的输入变量要将地址传给 scanf，无法由本次 pass 提升，故保留一条 alloca。这里的 IR 变量并非已经分配好的物理寄存器，后端仍须选择指令并决定哪些值驻留寄存器、哪些值溢出到栈。

（四） 优化等级与调试选项的对照

在 O0 的阶乘函数中，result、i 和 n 分别经 -8(%rbp)、-4(%rbp) 和 -20(%rbp) 访问。O2 的核心循环如下：

```
.L3:
  imull %eax, %edx
  addl  $1, %eax
  cmpl  %edi, %eax
  jne   .L3
```

O2 将循环值保存在寄存器中，并将边界整理为与 $n+1$ 比较；调试指令 .loc 仍用于关联源代码。用 nm -S 读取函数符号大小，用文件元数据读取整个可执行文件大小，结果见表3。

表 3: 同一源码的选项对照；大小单位为字节

构建方式	可执行文件大小	factorial 符号大小	含 .debug_info
O0、带 -g	17 704	54	是
O0、不带 -g	16 104	—	否
O2、带 -g	18 768	56	是
O0、带 -g、静态链接	1 035 896	—	是

O2 改善了循环的寄存器使用，但其函数大小略增，程序文件也更大；指令布局、对齐与调试信息都会影响体积。因此，不能从文件更小推断程序更快，也不能从 O2 名称推断本实验测得了加速。本实验未进行可信的性能计时，结论限于实际观察到的结构变化。

O0、O2、静态链接、原始 LLVM IR 和 mem2reg 后 IR 五种程序，分别验证 $0,\dots,12$、-1、13、非数字输入与空输入，共 $17\times 5=85$ 次检查，全部通过。非法输入无标准输出且返回 1，合法输入输出对应阶乘。另行验证宏修改后的结果为 60，避免把有意改变语义的变体纳入等价结论。

（五） 汇编、符号与链接

汇编器把目标相关汇编转换为机器码，建立节、符号表和重定位记录。readelf -h 显示 factorial.o 为 REL 类型；nm -u 显示尚未解析的 printf、__isoc99_scanf 和 __stack_chk_fail。最后一个符号源于本机 GCC 默认栈保护，与源码显式调用无关。

readelf -r 中，.rela.text 内偏移 0x68 和 0xa3 分别存在指向输入和输出函数的 R_X86_64_PLT32 重定位。汇编器已知道指令形式，但外部符号最终地址仍需链接器和动态装载机制处理。链接器组合目标文件、启动代码及库，解析符号、安排节并修正引用。默认链接程序带有动态解释器段；静态程序将所需库代码纳入文件，体积约为默认程序的 58.5 倍。两者在本实验输入上的输出一致，但依赖方式和产物布局不同。

四、 SysY、LLVM IR 与 RISC-V 等价编程

(一) 样例设计与语言特性

按 SysY 定义及文法补充 [2]，独立编写三组源程序、LLVM IR 和 RISC-V 汇编。手写 IR 与手写汇编保存在 `lab0/prep/02_equivalence/`，与工具自动生成的文件分开，便于检查表示之间的对应关系。表4列出覆盖情况。

表 4: 样例功能与语义覆盖

函数／样例	覆盖特性	可观察结果
factorial	while、赋值、乘法、比较、返回	$n!$
arithmetic	单目正负、加乘除模	$n + (n \bmod 3)$
control	if、break、continue	跳过 3，遇 9 停止累加
scope	嵌套块、常量、同名变量隐藏	$2n + 7$
logic	逻辑与或非、短路、全局副作用	合法输入时为 111
arrays_float	二维数组、数组形参、浮点加法、隐式转换	行求和、截断与混合运算
numeric_edges	负数除余、浮点四则与比较、递归、void 函数、局部数组补零与写入	商余、舍入、递归和、数组校验值

(二) 控制流、短路与作用域

`logic` 用 `bump()` 对全局计数器加一，使求值次数可以观察：

```
if ((n > 0 && bump() != 0) || bump() != 0)
    calls = calls + 10;
if (!(n < 0) && (n >= 0 || bump() != 0))
    calls = calls + 100;
```

对正数，第一次条件只执行左侧 `bump`；对 0，逻辑与跳过左侧调用，再执行逻辑或右侧。第二个条件在合法输入上不执行 `bump`，因此结果为 $1 + 10 + 100 = 111$。LLVM IR 通过条件 `br` 把函数调用放在对应路径中，汇编采用条件跳转实现同一控制流；若无条件计算逻辑运算的两个操作数，计数器就会暴露错误。

循环中的 `continue` 跳向下一次条件判断，`break` 跳到出口。LLVM 的 `phi` 在循环头合并不同前驱的循环变量和累计值。块作用域的同名变量则在名称解析后对应不同值：内层 `x=7` 不覆盖外层 `x=n` 的存储含义，故 `scope(n)` 返回 $2n + 7$。词法作用域与基本块属于不同层次，不能用基本块边界替代作用域管理。

(三) 整数语义与 RISC-V 调用约定

SysY 整数使用 32 位表示，手写 IR 采用 i32、sdiv 和 srem；RV64 汇编使用 addw、mulw、divw 和 remw 保持相应整数运算宽度。阶乘在手写汇编中的核心为：

```
    li t0, 1
    li t1, 1
.Lfactorial_loop:
    bgt t1, a0, .Lfactorial_exit
    mulw t0, t0, t1
    addiw t1, t1, 1
    j .Lfactorial_loop
.Lfactorial_exit:
    mv a0, t0
    ret
```

整数参数和返回值使用 a0 等参数寄存器；跨调用要保存的值放入 s0 等被调用者保存寄存器。非叶函数保存 ra 与实际使用的保存寄存器，栈保持 16 字节对齐。浮点参数和返回值遵循 LP64D，使用 fa0 等寄存器 [7]。正确的局部指令序列若破坏调用约定，仍可能在运行库调用后出现错误。

(四) 数组地址与浮点转换

数组样例定义 weights[2][2]={{1.5,2.0},{3.25,4.0}}，通过形参 float values[][2] 传入行求和函数。元素 values[row][col] 的字节偏移为

$$\text{offset} = 4(2\,row + col),$$

因为每个单精度元素占 4 字节、每行两个元素。LLVM IR 用 getelementptr 按类型计算地址；手写汇编将行号乘 8，再用 flw 读取两列并以 fadd.s 相加。

浮点和赋给整数时，LLVM 使用 fptosi，汇编使用带向零舍入模式的 fcvt.w.s ..., rtz。整数行号参与浮点加法时，分别使用 sitofp 和 fcvt.s.w。两行的和为 3.5 与 7.25，截断后为 3 与 7；加入行号后输出浮点值 3.5 与 8.25。课程运行库的 putfloat 以十六进制浮点格式打印，分别对应 0x1.cp+1 与 0x1.08p+3。这种输出格式是运行库约定，不是数值计算错误。

以下为手写 row_sum 的 LLVM IR 主体；它对应源码的 values[row][0]+values[row][1]，没有省略实际的数组读取：

```
%index = sext i32 %row to i64
%left.ptr = getelementptr [2 x float], ptr %values, i64 %index, i64 0
%right.ptr = getelementptr [2 x float], ptr %values, i64 %index, i64 1
%left = load float, ptr %left.ptr, align 4
%right = load float, ptr %right.ptr, align 4
%sum = fadd float %left, %right
ret float %sum
```

getelementptr 只计算地址，load 才读取元素。形参首维退化为指针，故 GEP 的源元素类型是“一行”的 [2 x float]。汇编用 slli t0,a1,3 计算行偏移，两个 flw 访问偏移 0 和 4，fadd.s 将结果放入 fa0 返回；三种表示因此具有相同的行跨度、元素宽度和返回类型。

（五） 负数运算、递归与数组写入的补充验证

只使用非负数不能区分向零取整与向下取整。`numeric_edges` 读取 n, d, 限制 $-12 \le n \le 12$、$1 \le |d| \le 3$，在除法之前拒绝零除数和越界值。按语言规则，$q = \mathrm{trunc}(n/d)$、$r = n - qd$；因此 $-7/3 = -2$、$-7 \bmod 3 = -1$，而 $7/(-3) = -2$、$7 \bmod (-3) = 1$。若测试预期直接采用 Python 的 `//` 和 `%`，会误用向下取整规则。本实验在独立 JSON 中显式保存这些预期值。

浮点函数计算 $y = (((-x/2) - 0.25) + 1) \times 2/4$，其中 x 由 n 转换而来。手写 IR 依次使用 `fneg/fdiv/fsub/fadd/fmul/fdiv`，汇编使用对应单精度指令；$n = 7$ 时 $y = -1.375$，转为整数应为 -1。另外以浮点小于比较输出符号标记。递归函数计算 $S(k) = k + S(k-1)$、$S(0) = 0$，令 $k = |n|$，检查多层调用时返回地址和保存寄存器是否保持正确。

局部数组声明为 `int values[3]={n}`，后两个元素必须补零；`void update(int values[],int delta)` 通过数组形参读写后，数组应为 $(n, d, n - d)$。输出加权校验值 $v_0 + 2v_1 + 3v_2 = 4n - d$，使缺失初始化或错误写入更易被观察。该样例共有 15 个输入，包括商余的四种符号组合、递归基例、正负范围端点和五个非法输入。

（六） 链接路径及结果

五条路径在每个用例上分别执行，结果见表5。其中“源程序”路径将本实验 SysY 子集按 C 语法编译；“独立 IR”路径使用手写 `.ll`；“手写汇编”路径直接汇编链接。

表 5: 五条执行路径的等价比对结果

执行路径	控制流	数组	数值边界	合计
源程序 → Clang → 宿主程序	15/15	4/4	15/15	34/34
独立 IR → Clang → 宿主程序	15/15	4/4	15/15	34/34
手写汇编 → 交叉链接 → QEMU	15/15	4/4	15/15	34/34
源程序 → RISC-V GCC → QEMU	15/15	4/4	15/15	34/34
独立 IR → llc → 链接 → QEMU	15/15	4/4	15/15	34/34
全部	75/75	20/20	75/75	170/170

`features` 输入 $-1, 0, \ldots, 12, 13$；`arrays_float` 输入 $-1, 0, 1, 2$。越界输入输出 `-1`，且在乘法或数组访存之前退出。典型输出见表6，每个单元格对应一行标准输出。

表 6: 代表性输入与五条路径一致的输出

样例	输入	输出
features	0	`1 0 0 7 111`
features	5	`120 7 12 17 111`
features	12	`479001600 12 33 31 111`
features	13	`-1`
arrays_float	0	`0x1.cp+1 3`
arrays_float	1	`0x1.08p+3 7`
arrays_float	2	`-1`

数值边界样例按“商、余数、浮点结果、截断值、负数标记、递归和、数组校验值”输出。例如输入 `7 -3` 得到 `-2 1 -0x1.6p+0 -1 1 28 31`。所有路径都链接课程 SysY 运行库，退出码均为 0；非法输入按样例约定输出 `-1`。测试覆盖三组程序的正常路径和边界，尚未穷举全部 SysY 语法、NaN ／无穷值或任意浮点输入；本实验也不将整数溢出、除零或越界访存作为合法算术结果比较。

五、 MLIR 与 AscendNPU IR 探索

(一) 通用 VecAdd 的逐层降低

MLIR 允许在同一框架中保留不同抽象层级的操作和类型,并通过转换 pass 逐步引入目标约束 [8]。本地设计四元素单精度向量加,起始表示含 `scf.for`、`arith.addf` 及 `memref.load/store`。输入

$$a = (1, 2, 3, 4), \qquad b = (0.5, 2.5, 4.5, 6.5),$$

期望输出为 $(1.5, 4.5, 7.5, 10.5)$。

`convert-scf-to-cf` 将结构化循环转换为显式控制流，实际输出的循环头如下:

```
cf.br ^bb1(%c0 : index)
^bb1(%0: index):
  %1 = arith.cmpi slt, %0, %c4 : index
  cf.cond_br %1, ^bb2, ^bb3
```

循环变量成为块参数，回边通过 `cf.br` 传入更新值；其数据流作用对应 LLVM 中 `phi` 的合并机制。后续转换将 MemRef、Arith、Func 和 CF 依次降至 LLVM 方言，并消除转换过程中遗留的类型桥接操作。核心命令为:

```
mlir-opt-15 01_structured.mlir --convert-scf-to-cf \
  -o 02_cfg.mlir
mlir-opt-15 02_cfg.mlir --convert-memref-to-llvm \
  --convert-arith-to-llvm --convert-func-to-llvm \
  --convert-cf-to-llvm --reconcile-unrealized-casts \
  -o 03_llvm_dialect.mlir
mlir-translate-15 --mlir-to-llvmir \
  03_llvm_dialect.mlir -o 04_vecadd.ll
```

MemRef 降低后需要显式表达指针、偏移、维度和步长。实验通过 `llvm.emit_c_interface` 生成 C 接口，以包含 allocated/aligned 指针、offset、sizes 和 strides 的一维描述符传入数组。这里的静态类型 `memref<4xf32>` 要求 offset 为 0、size 为 4、stride 为 1；描述符含有这些字段并不表示该函数支持任意步长。三个 MemRef 在内部函数签名中展开为 15 个参数，外层 C 包装函数仍接收三个描述符指针。

表7以保存的转储对照同一循环。LLVM 方言的 `llvm.fadd` 仍是 MLIR 操作；经 `mlir-translate` 后才成为 LLVM IR 的 `fadd` 指令，两者不能只因名称相近便混为同一表示。

表 7: VecAdd 核心循环在各层的实测变化

阶段	控制流与数据表示	数值／内存操作
结构化 MLIR	`scf.for`、`index` 循环变量	2 次 `memref.load`、1 次加法、1 次写回
CF 层	`cf.cond_br` 和块参数	MemRef 访存及 `arith.addf` 保留
LLVM 方言	`llvm.cond_br`、`i64` 索引、描述符	3 个 GEP、2 个 load、1 个 `llvm.fadd`、1 个 store
LLVM IR	`br`、`phi i64` 和指针	`getelementptr/load/fadd/store`

最终 IR 中 `%32 = phi i64 [ %44, %34 ], [ 0, %15 ]` 接收初值和回边值；循环体的 `%41 = fadd float %37, %40` 对应起始的 `arith.addf`。加法本身保持一一对应，新增操作主要承担控制流、地址计算和 ABI 适配，故降低过程不能仅按 IR 行数评价优劣。

最终 LLVM IR 经 verifier 检查，再由 Clang 与 C 驱动链接。三组逐元素检查分别得到：基本输入 $(1.5, 4.5, 7.5, 10.5)$；正负混合输入 $(-1, -2, 0, -0.125)$；将输出与第一输入设为同一数组时仍得到后一结果，共 $3 \times 4 = 12$ 个元素通过。原地测试只覆盖两者完全重合，不扩展为任意部分重叠的保证。各数值均可由二进制精确表示，故此处使用等值比较。

(二) 官方 Ascend VecAdd 与方言分工

课程进阶要求允许在环境配置困难时通过阅读文档说明探索过程。实验据此阅读课程课件与官方 VecAdd 页面 [4, 9, 11]，在 `lab0/prep/03_mlir/ascend_vecadd.mlir` 保留注明来源的参考程序。

官方示例处理 16 个 `i16` 元素。参数位于 GM 地址空间，两个 `hivm.hir.load` 将输入搬入 UB，`hivm.hir.vadd` 在片上完成加法，再由 `hivm.hir.store` 写回 GM。`hacc.entry` 与 DEVICE 属性表达设备入口。该程序已经包含硬件相关操作，不能把它称为未经降低的通用高层算子。

表 8: 课程材料中的 AscendNPU IR 分工

表示	主要表达内容	VecAdd 探索中的关注点
HFusion	较高层算子及融合相关信息	在结构尚保留时进行变换与融合
HIVM	设备操作、地址空间、搬运及同步约束	GM/UB、load/vadd/store
HACC	异构执行、设备入口与函数属性	entry、DEVICE 与主机／设备协作

这些方言承载不同职责，可在表示中协作；HACC 不是简单排在 HIVM 之后、替换全部算子的单一流水级。逐层降低让高层优化保留所需语义，再逐步落到存储、指令和硬件资源约束。

(三) 从官方示例推导存储与同步约束

为使调研落到具体程序，沿官方例子的两次 load、一次 vadd、一次 store 追踪数据依赖，并与架构文档中的核内存储和流水映射职责对照 [10]。三个 UB 缓冲区的逻辑载荷各为 $16 \times 2 = 32$ 字节，合计 96 字节；GM 的逻辑读写量为 $2 \times 32 + 32 = 96$ 字节。这只是按元素数推导的有效载荷，实际分配可能包含对齐、临时空间或复用，不能冒充内存规划 pass 的实测结果。

图2给出需要保持的偏序：向量加必须等两个输入搬入 UB 后才可执行，写回必须等待加法结果。该图表达数据依赖，不指定真实事件号或同步指令；专用编译器还需将依赖映射到硬件流水。



图 2: 由官方 VecAdd 源码推导的数据依赖；非设备实测时间线

这个小样例只有一个向量加，没有相邻算子可融合，也不含 Cube 运算，因而不能用它声称已验证算子融合收益或 Cube/Vector 混合核拆分。它适合观察地址空间、搬运、内存规划和同步约束何时进入 IR。课件中的地址偏移、同步伪码与 `llvm.npu.*` 名称用于解释原理；本报告没有将其作为特定版本的真实 pass 输出。

官方设备端编译方式为：

```
bishengir-compile add.mlir -enable-hivm-compile -o kernel.o
```

随后由 CANN host 程序加载或注册设备二进制、启动 kernel 并取回结果。本次环境的 PATH 中未找到 `bishengir-compile`、`bishengir-opt`、`hivmc` 或 `npu-smi`，默认 CANN 安装目录也不存在。实际用 `mlir-opt-15` 读取参考程序时，工具在 `hivm.address_space` 处报告 `unregistered dialect` 并返回 1，诊断保存在 `lab0/report/evidence/ascend-probe.json`。允许未注册方言只能绕过相应解析限制，不能补上缺失的验证器和 lowering pass；因此未生成设备二进制，也未采集专用 pass 转储。

阅读官方 host 代码时还发现，其比较循环打印 `Expect` 和 `Result`，随后直接打印成功提示；循环本身未因数值不等而报错。因此，未来上板验证应检查全部 16 个元素相等并在失败时返回非零值，同时检查运行时调用结果，不能把示例中的成功字符串单独当作正确性证据。通用 MLIR 的 C 驱动已经采用逐元素检查并返回失败状态。

(四) 比较与探索边界

通用实验明确展示了“结构化循环 → 显式 CFG → LLVM 方言 → LLVM IR”的实测过程。Ascend 文档则补充了 CPU 样例中不显式出现的片上存储、数据搬运和设备入口。前者验证逐层转换后的数值结果，后者用于理解专用方言的设计。两部分共同说明多级表示的价值，但通用实验通过并不能证明 Ascend 参考程序在 NPU 上通过；设备专用调度、同步和性能仍需相应工具链与硬件验证。

六、　结论

本实验通过阶段产物建立了源程序与目标程序之间可检查的联系。预处理改变编译输入，前端组织并检查程序语义，中端将控制流及数据流显式化并优化，后端结合目标指令和调用约定生成汇编，汇编器与链接器再完成机器码、符号与地址的组织。

等价编程的 170 次比对和编译流程的 85 次检查均通过。短路副作用、二维数组寻址、递归调用、负数除余和浮点舍入，是低层实现中需要维护的语义；SSA 提升与 O2 对照则表明，正确性、指令结构、文件体积和性能是不同问题。MLIR 四级转储及 12 个输出元素验证说明逐层转换如何保留计算；Ascend 调研则进一步把向量加映射到地址空间、搬运和同步依赖。后续课程编译器可复用这些样例与运行环境，逐步实现语法分析、类型检查、SSA、优化及自有 RISC-V 后端。

附录 A　复现与结果文件

在 Ubuntu ／ WSL 安装表1中的工具，并将课程运行库放入 `lab0/runtime/lib.tar.gz` 后，在仓库根目录执行:

```
make prep       # compilation stages, 170 equivalence checks, MLIR
make test       # current compiler lexer regression tests
make report     # export PDF from lab0/report/preparation.tex
make clean      # remove rebuildable intermediates
```

`lab0/report/evidence/results.json` 保存执行时间、工具版本、源码及运行库 SHA-256、命令、输入和实际输出；`lab0/report/evidence/observations.txt` 保存报告使用的 GIMPLE、IR、汇编、诊断及 MLIR 各层输出。`lab0/build/` 中的完整日志与二进制可重新生成，清理不会删除上述报告依据。词法分析器测试仅验证当前工程状态，不计入本次预备实验的 170 次等价比对。

参考文献

[1] 课程资料. 预备工作：了解你的编译器，上机作业要求与作业完成要求.

[2] 课程资料. SysY2022 语言定义 V1；SysY 文法补充说明.

[3] 课程资料. SysY2022 运行时库 V1；lib.tar.gz.

[4] 课程资料. 编译器开发环境部署；了解编译器与 LLVM IR 编程；AscendNPU-IR 课件.

[5] GNU Project. GCC 12.3 Manual: Options Controlling the Kind of Output. https://gcc.gnu.org/onlinedocs/gcc-12.3.0/gcc/Overall-Options.html.

[6] LLVM Project. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html.

[7] RISC-V International. RISC-V ABIs Specification. https://riscv-non-isa.github.io/riscv-elf-psabi-doc/.

[8] LLVM Project. MLIR Documentation. https://mlir.llvm.org/docs/.

[9] Huawei. AscendNPU IR：编译与执行示例. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/quick_start/examples_zh.html. 阅读日期：2026 年 10 月 10 日.

[10] Huawei. AscendNPU IR：架构设计. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/architecture_zh.html. 阅读日期：2026 年 10 月 10 日.

[11] Huawei. 毕昇编译器与 CANN. https://www.hiascend.com/cann/bisheng.